

NITHUB CTF // WALKTHROUGH

HANDS-ON GUIDE :: RECON → SQLi → IDOR → PRIV-ESC

SETUP ARM YOUR TOOLS

Parrot OS ships with everything. Launch Burp Suite:

```
$ burpsuite
```

Temporary project → Start Burp. Then **Proxy → Open Browser** – Burp's built-in browser is pre-proxied, zero config. Use it and skip the Firefox cert dance.

Replace **TARGET** below with the link the host shares.

```
# example
TARGET = https://your-link.trycloudflare.com
```

RECON STAGE 1 — FIND THE HIDDEN PATH

Method A – the human way: just read robots.txt.

```
$ curl -s $TARGET/robots.txt
```

Output reveals **Disallow: /dev-backup**. robots.txt lists what the owner doesn't want crawled – a gift-wrapped map of interesting paths.

Method B – the scanner way: fuzz for directories. If a player skips robots.txt, directory brute-forcing finds the same paths. Both tools are on Parrot:

```
# ffuf — fast, show only real hits
$ ffuf -u $TARGET/FUZZ -w /usr/share/wordlists/dirb/common.txt -mc 200,301,302
```

```
# gobuster — same idea
$ gobuster dir -u $TARGET -w /usr/share/wordlists/dirb/common.txt
```

WHY: the **FUZZ** keyword is where ffuf injects each wordlist entry. **-mc 200,301,302** means "only show me paths that actually exist." You'll see /portal, /dev-backup, /admin surface. Recon = mapping the attack surface.

LOOT STAGE 2 — READ THE LEAKED FILES

1. Visit **\$TARGET/dev-backup** – directory listing is left ON.
2. Open **config.php.bak** – comments admit the login query is built by gluing strings (injectable).
3. Open **notes.txt** – confirms 3 weaknesses: concatenated SQL login, an **?id=** with no ownership check, and a salary gated by a **client-side role** cookie.

WHY: those three notes ARE the roadmap for the whole box. Info disclosure is finding secrets the app left lying around.

EXPLOIT STAGE 3 — SQLI LOGIN BYPASS (BURP REPEATER)

1. At **\$TARGET/portal**, submit any username/password once.
2. Burp → **Proxy** → **HTTP history**, find the POST to /portal → right-click → **Send to Repeater**.
3. In **Repeater**, change the username value and send:

```
username=' OR '1'='1' -- &password=x
```

Response flips from "Invalid credentials" to a **302 redirect** to /profile?id=1001, setting cookies **session=ok** and **role=employee**.

WHY: the query becomes ... **WHERE user='' OR '1'='1' -- '.... '1'='1'** is always true and **--** comments out the rest. Auth bypassed.

PIVOT STAGE 4 — IDOR TO THE CEO (BURP INTERCEPT)

1. You're now employee id **1001**. Visit **\$TARGET/profile?id=1001**.
2. Burp → **Proxy** → **Intercept ON**.
3. In the browser change the URL to **?id=1002** and hit enter – request pauses in Burp.
4. Edit the id right in the intercepted request. Walk them: 1002, 1003, 1004 → **1005**.
5. **Forward** each. id **1005** = **The CEO**. Salary shows REDACTED / admin-only.

Turn Intercept OFF after, or the browser will seem to hang.

WHY: IDOR – the server hands you any record id you ask for without checking it's yours.

PRIV-ESC STAGE 5 — ROLE COOKIE TAMPERING

Method A – Burp: Intercept ON → go to /admin/payroll → find the Cookie header → edit:

```
Cookie: session=ok; role=admin
```

Method B – DevTools: F12 → Application/Storage → Cookies → change **role** from **employee** to **admin** → reload.

WHY: never trust what the client can edit. Role checks belong on the server, not in a cookie.

FLAG STAGE 6 — CAPTURE & SUBMIT

With **role=admin**, **\$TARGET/admin/payroll** loads the executive payroll. CEO salary = **\$450,000**. Grab the flag:

```
flag{NITHUB_450k_USD}
```

Submit at **\$TARGET/submit**. Your **SOLVED** badge lights up on the scoreboard.